

Autoregressive Integrated Moving Average Process (ARIMA)

Course: Practical Time Series Analysis

Lecturer: Dr. David Zmiaikou

May 21, 2024

Contents

01 Autoregressive Moving Average Process

02 Hourly Bandwidth Usage (Implement)

03 Hourly Bandwidth Usage (Forecast)

04 Autoregressive Integrated Moving Average Process (ARIMA)

01

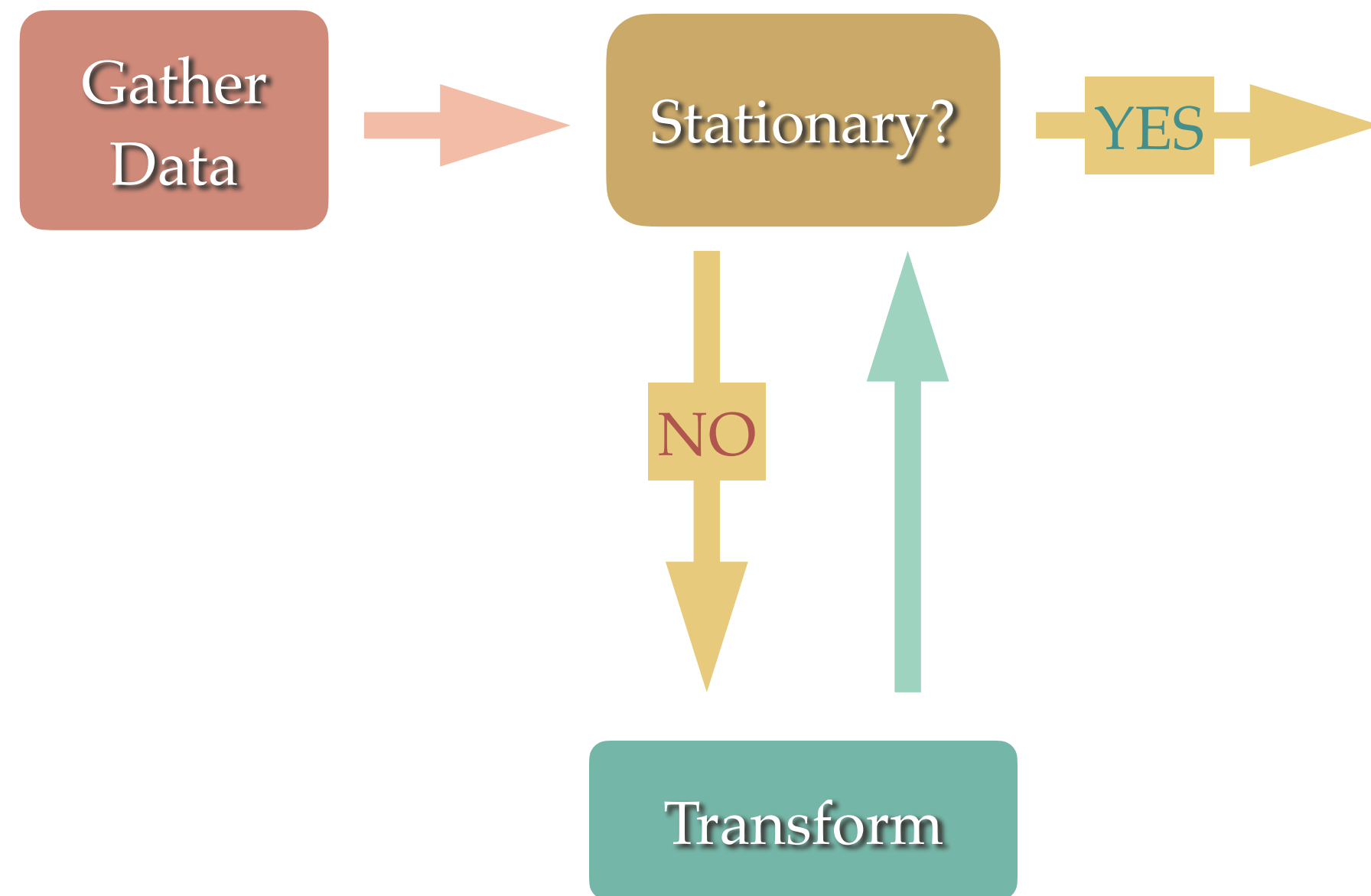
Autoregressive Moving Average Process (ARMA)

^{def} What is an Autoregressive Moving Average Process (ARMA)?

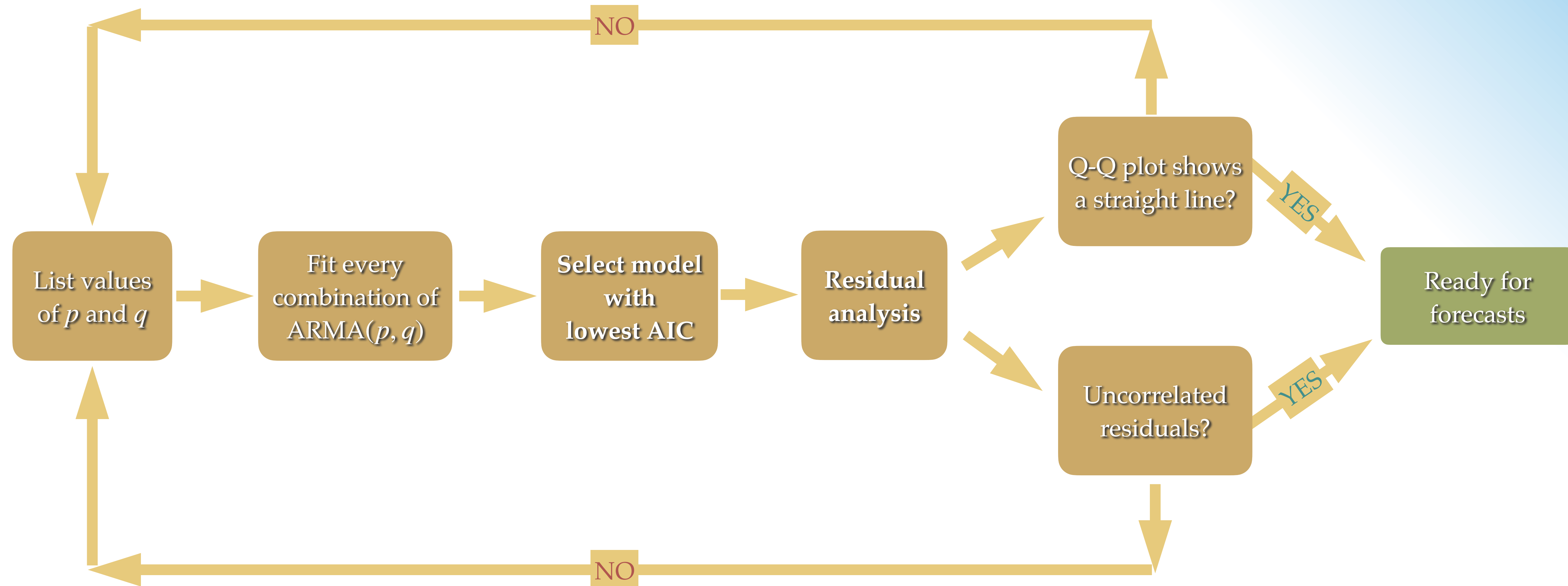
- The **autoregressive moving average process** is a combination of the autoregressive process and the moving average process.
- It is denoted by $ARMA(p, q)$, where p is the **order** of the autoregressive part, and q is the **order** of the moving average part. The general expression is

$$y_t = C + \phi_1 y_{t-1} + \phi_2 y_{t-2} + \cdots + \phi_p y_{t-p} + \mu + \epsilon_t + \theta_1 \epsilon_{t-1} + \theta_2 \epsilon_{t-2} + \cdots + \theta_q \epsilon_{t-q}$$

Identifying an Autoregressive Moving Average Process



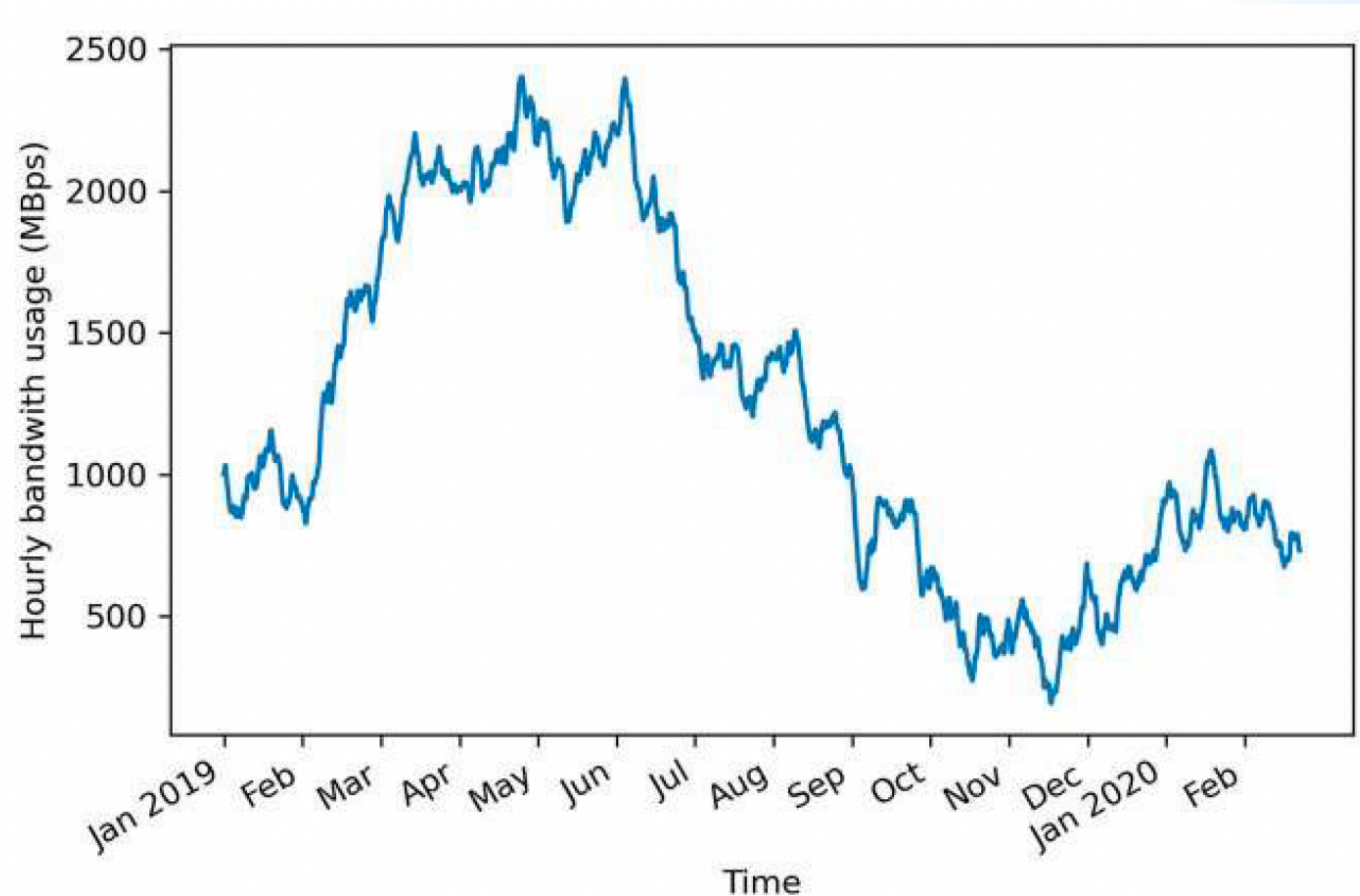
Identifying an Autoregressive Moving Average Process



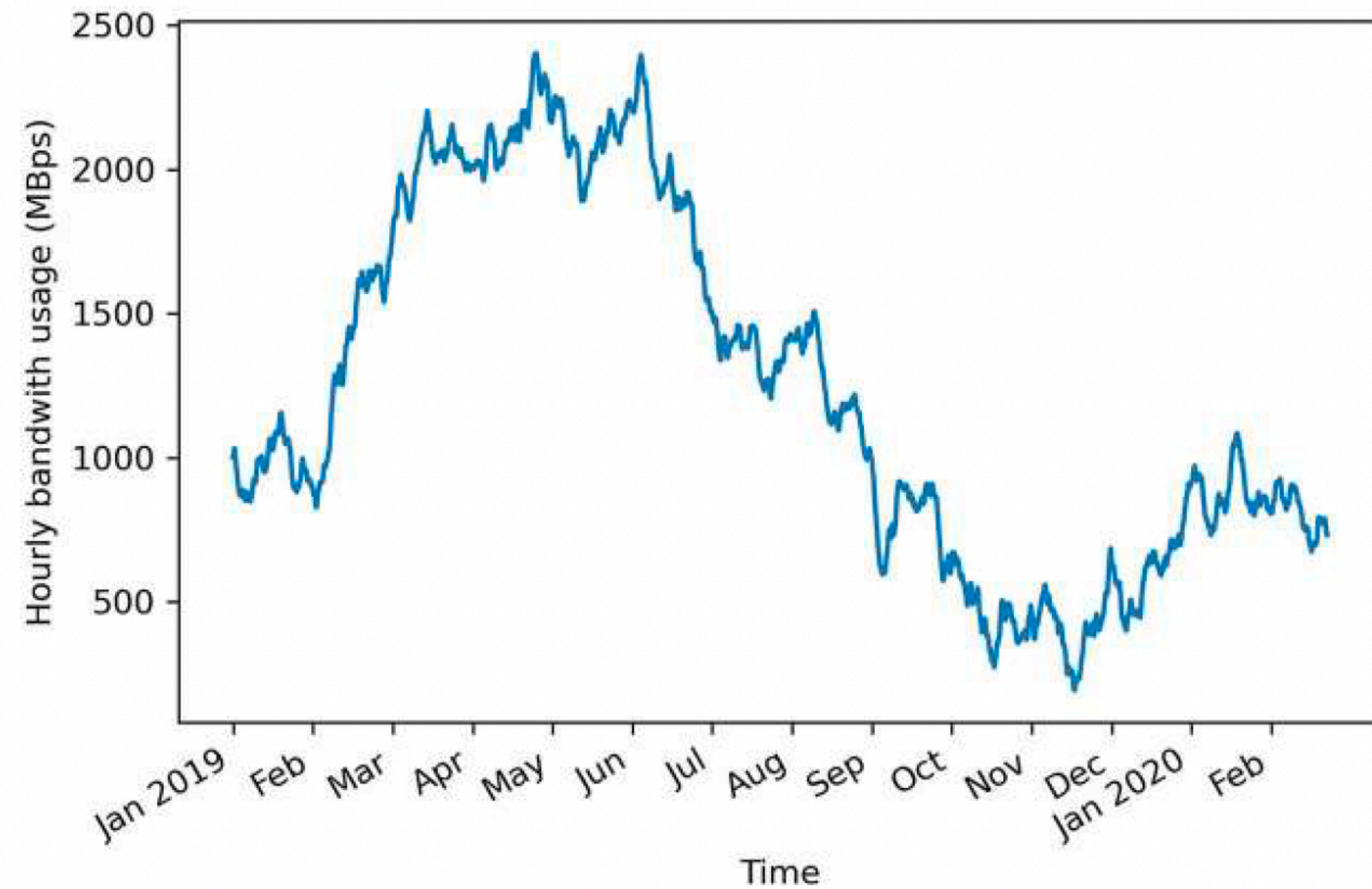
02

Hourly Bandwidth Usage (Implement)

Example 1: Hourly bandwidth usage in a data centre since January 1, 2019 (10,000 points)



Hourly bandwidth usage: Stationarity



- We will use **DataFrame** from **pandas**:

```
import pandas as pd  
df = pd.read_csv('bandwidth.csv')
```

- Use the **adfuller** function from **statsmodels**:

```
from statsmodels.tsa.stattools import adfuller  
ADF_result = adfuller(df['hourly_bandwidth'])  
print(f'ADF Statistic: {ADF_result[0]}')  
print(f'p-value: {ADF_result[1]}')
```

This results in an ADF statistic of -0.8 and a p-value of 0.80 . *Not stationary.*

- Try to de-trend by applying a first-order differencing. Use the **diff** method from the **numpy** library:

```
import numpy as np  
bandwidth_diff = np.diff(df['bandwidth_diff'], n=1)
```

This results in an ADF statistic of -20.69 and a p-value of 0.0 . *Stationary!*

def What is Akaike information criterion (AIC)?

Number of model
parameters

$$AIC = 2\ln\left(\frac{e^k}{\hat{\mathcal{L}}}\right)$$

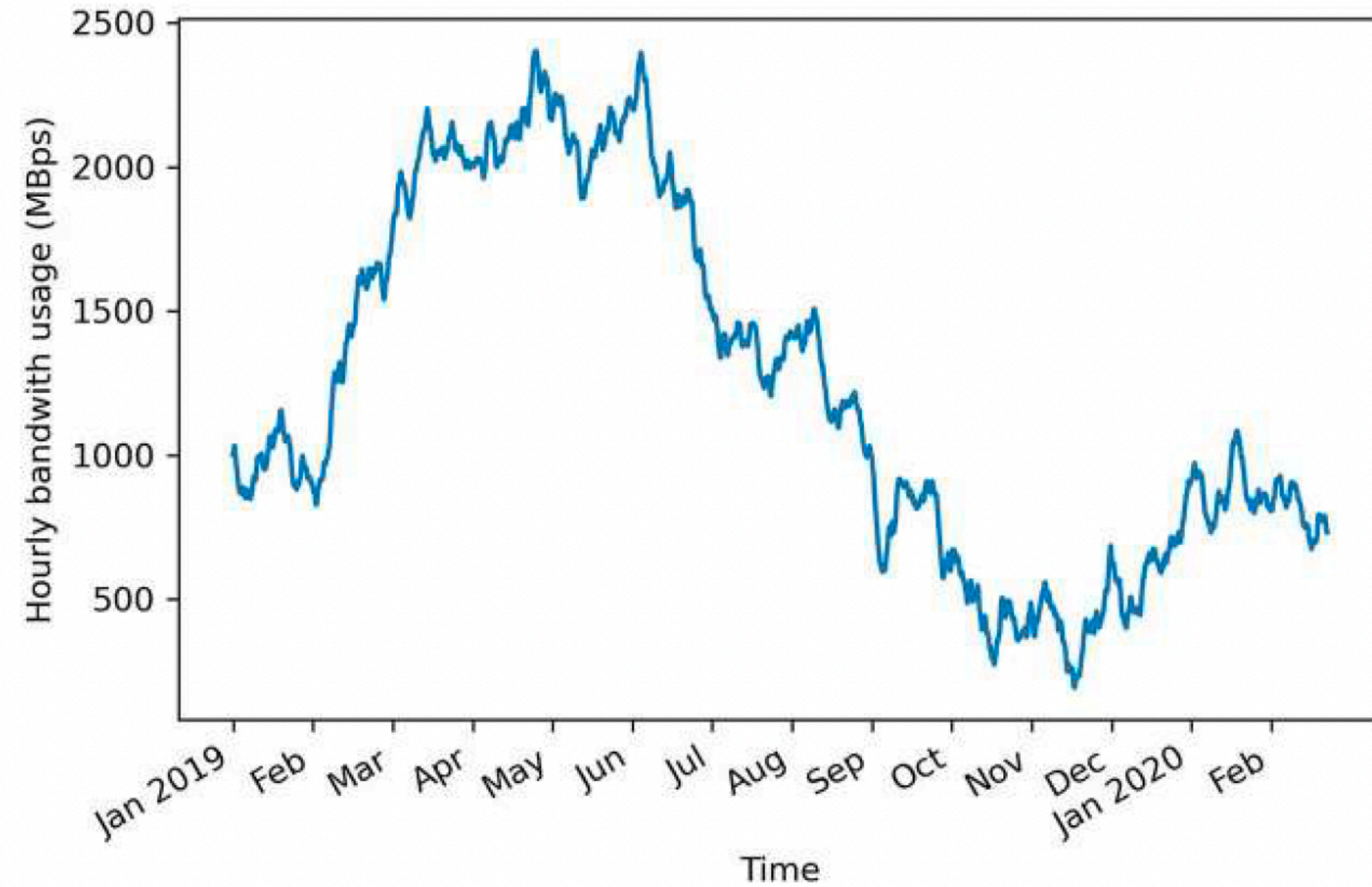
Maximized Likelihood

- The **Akaike information criterion (AIC)** is a measure of the quality of a model in relation to other models. It is used for model selection.
- The AIC is a function of the number of parameters k in a model and the maximum value of the likelihood function $\hat{\mathcal{L}}$:

$$AIC = 2k - 2\ln(\hat{\mathcal{L}})$$

- The lower the value of the AIC, the better the model. Selecting according to the AIC allows us to keep a balance between the complexity of a model and its goodness of fit to the data.

Hourly bandwidth usage: AIC



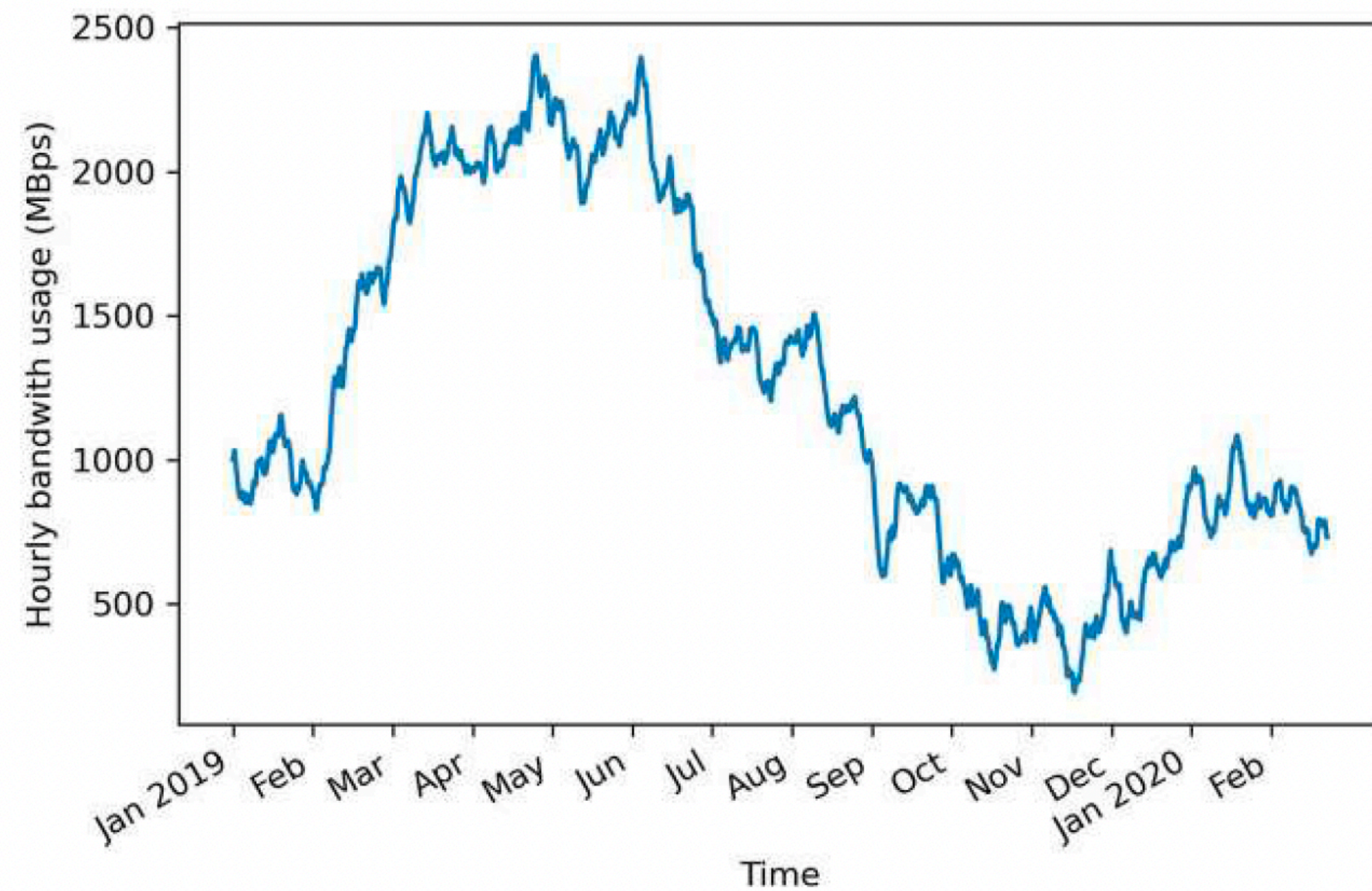
- Allow the values of p and q to vary from 0 to 3 (this range is arbitrary). Create a list of all possible combinations of (p, q) , using the **product** function from **itertools**:

from itertools import product

ps = range(0, 4, 1)

qs = range(0, 4, 1)

order_list = list(product(ps, qs))



Hourly bandwidth usage: AIC

- Fit all unique ARMA(p, q) models:

```
from typing import Union  
from tqdm import tqdm_notebook  
from statsmodels.tsa.statespace.sarimax import SARIMAX
```

```
def optimize_ARMA(endog: Union[pd.Series, list], order_list: list) -> pd.DataFrame:  
    results = []  
    for order in tqdm_notebook(order_list):  
        try:  
            model = SARIMAX(endog, order=(order[0], 0, order[1]),  
                           simple_differencing=False).fit(dispatch=False)  
        except:  
            continue  
        aic = model.aic  
        results.append([order, aic])  
    result_df = pd.DataFrame(results)  
    result_df.columns = ['(p,q)', 'AIC']  
  
    #Sort in ascending order, lower AIC is better  
    result_df = result_df.sort_values(by='AIC',  
                                     ascending=True).reset_index(drop=True)  
    return result_df
```

Hourly bandwidth usage: AIC

	(p,q)	AIC
0	(3, 2)	27991.063879
1	(2, 3)	27991.287509
2	(2, 2)	27991.603598
3	(3, 3)	27993.416924
4	(1, 3)	28003.349550
5	(1, 2)	28051.351401
6	(3, 1)	28071.155496
7	(3, 0)	28095.618186
8	(2, 1)	28097.250766
9	(2, 0)	28098.407664
10	(1, 1)	28172.510044
11	(1, 0)	28941.056983
12	(0, 3)	31355.802141
13	(0, 2)	33531.179284
14	(0, 1)	39402.269523
15	(0, 0)	49035.184224

- Fit all unique $\text{ARMA}(p, q)$ models:

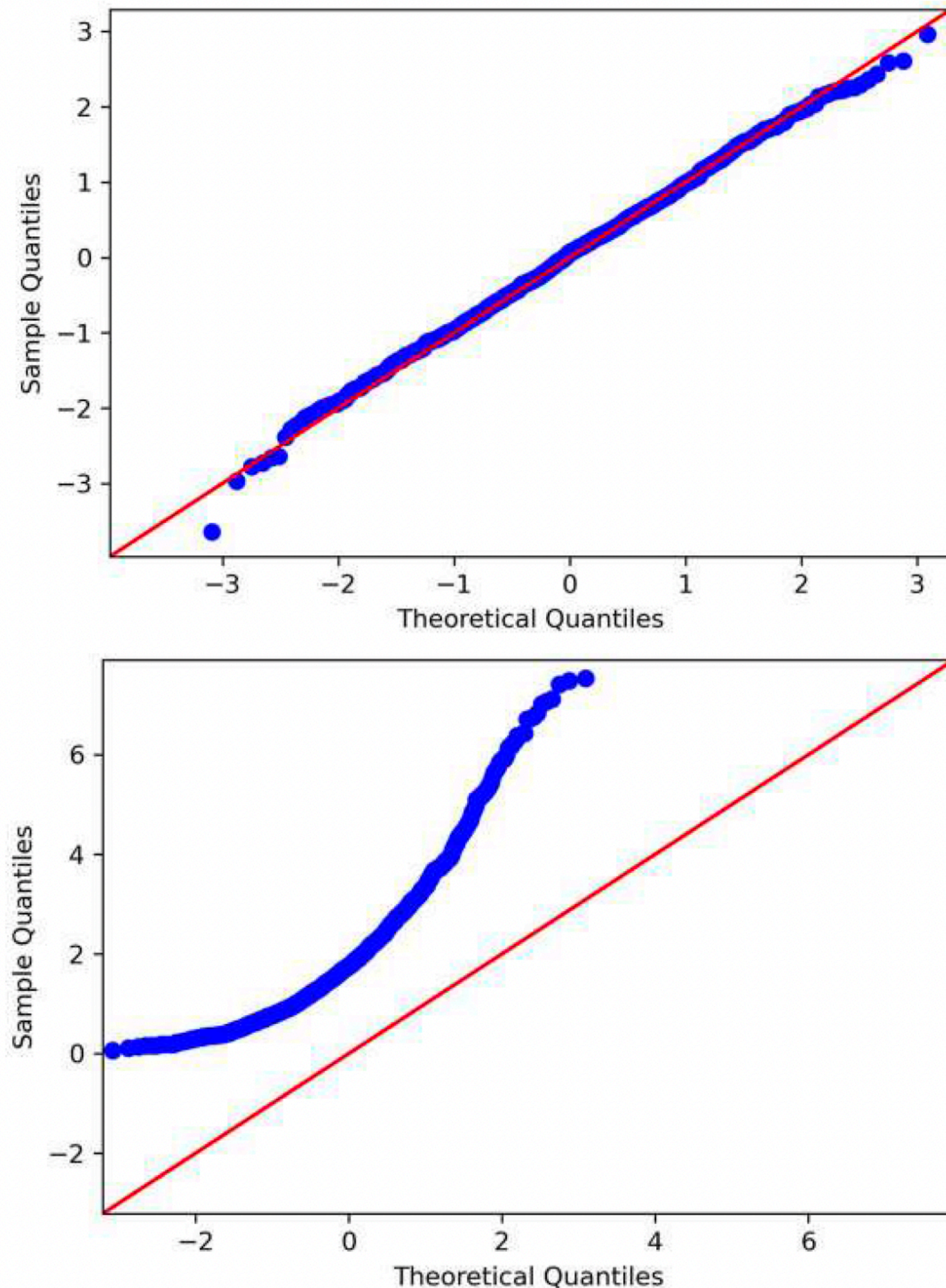
result_df = optimize_ARMA(train['bandwidth_diff'], order_list)

Hourly bandwidth usage: AIC

	(p,q)	AIC
0	(3, 2)	27991.063879
1	(2, 3)	27991.287509
2	(2, 2)	27991.603598
3	(3, 3)	27993.416924
4	(1, 3)	28003.349550
5	(1, 2)	28051.351401
6	(3, 1)	28071.155496
7	(3, 0)	28095.618186
8	(2, 1)	28097.250766
9	(2, 0)	28098.407664
10	(1, 1)	28172.510044
11	(1, 0)	28941.056983
12	(0, 3)	31355.802141
13	(0, 2)	33531.179284
14	(0, 1)	39402.269523
15	(0, 0)	49035.184224

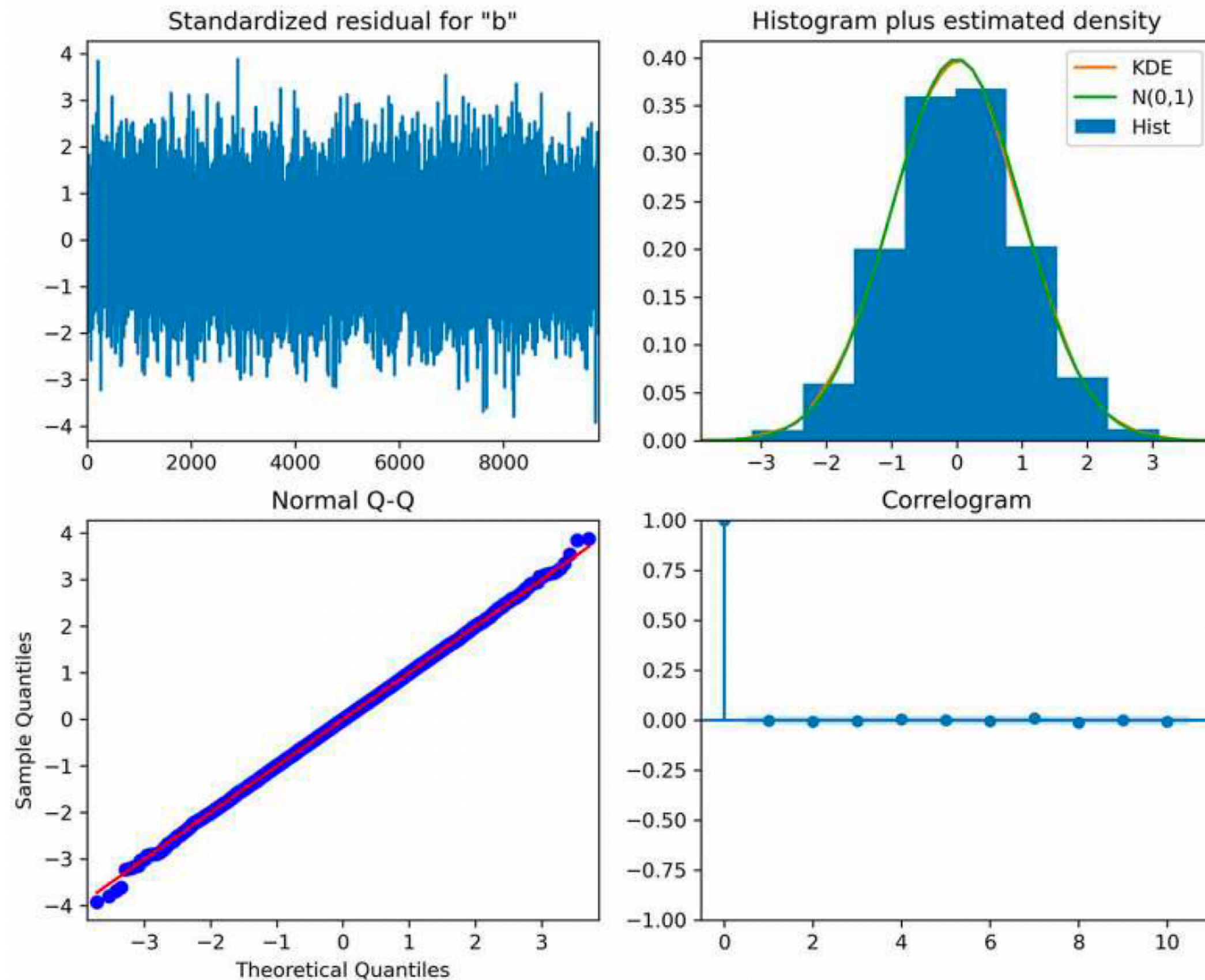
- The first three models all have an AIC of 27,991, with only slight differences. Therefore, one would argue that the ARMA(2,2) model is the model that should be selected. Its AIC value is very close to the ARMA(3,2) and ARMA(2,3) models, while being less complex, since it has four parameters to be estimated instead of five.
- Therefore, we'll select the ARMA(2,2) model and move on to the next steps, which is the analysis of the model's residuals.

Residual Analysis: Q-Q plot



- The **quantile-quantile plot (Q-Q plot)** is a graphical tool for verifying our hypothesis that the model's residuals are normally distributed.
- The Q-Q plot is constructed by plotting the quantiles of our residuals on the y -axis against the quantiles of a theoretical distribution, in this case the normal distribution, on the x -axis.
- If both distributions are similar, meaning that the distribution of the residuals is close to a normal distribution, the Q-Q plot will display a straight line that approximately lies on $y = x$. This in turn means that our model is a good fit for our data.

Hourly bandwidth usage: Q-Q plot

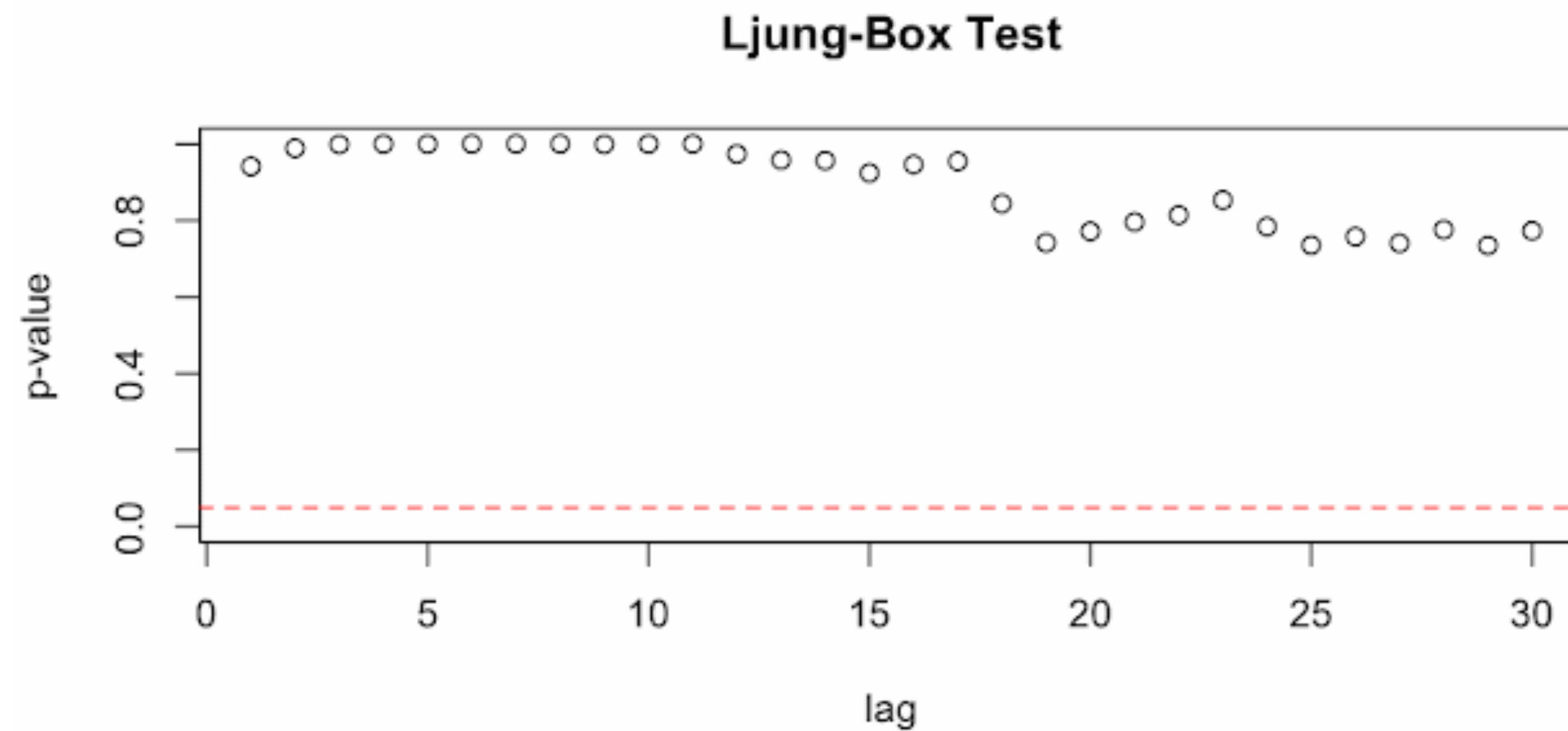


- Use the **plot_diagnostics** method to study the Q-Q plot, as well as the other accompanying plots:

```
model = SARIMAX(train['bandwidth_diff'],  
order=(2,0,2), simple_differencing=False)
```

```
model_fit = model.fit(dis=False)  
model_fit.plot_diagnostics(figsize=(10, 8));
```

Residual Analysis: Ljung-Box Test



- The **Ljung-Box test** is a statistical test that determines whether the autocorrelation of a group of data is significantly different from 0.
- We apply the Ljung-Box test on the model's residuals to test whether they are similar to white noise.
- The *null hypothesis* states that there is no autocorrelation.
- If the **p-value is larger than 0.05**, then the residuals are independently distributed.

Hourly bandwidth usage: Ljung-Box Test

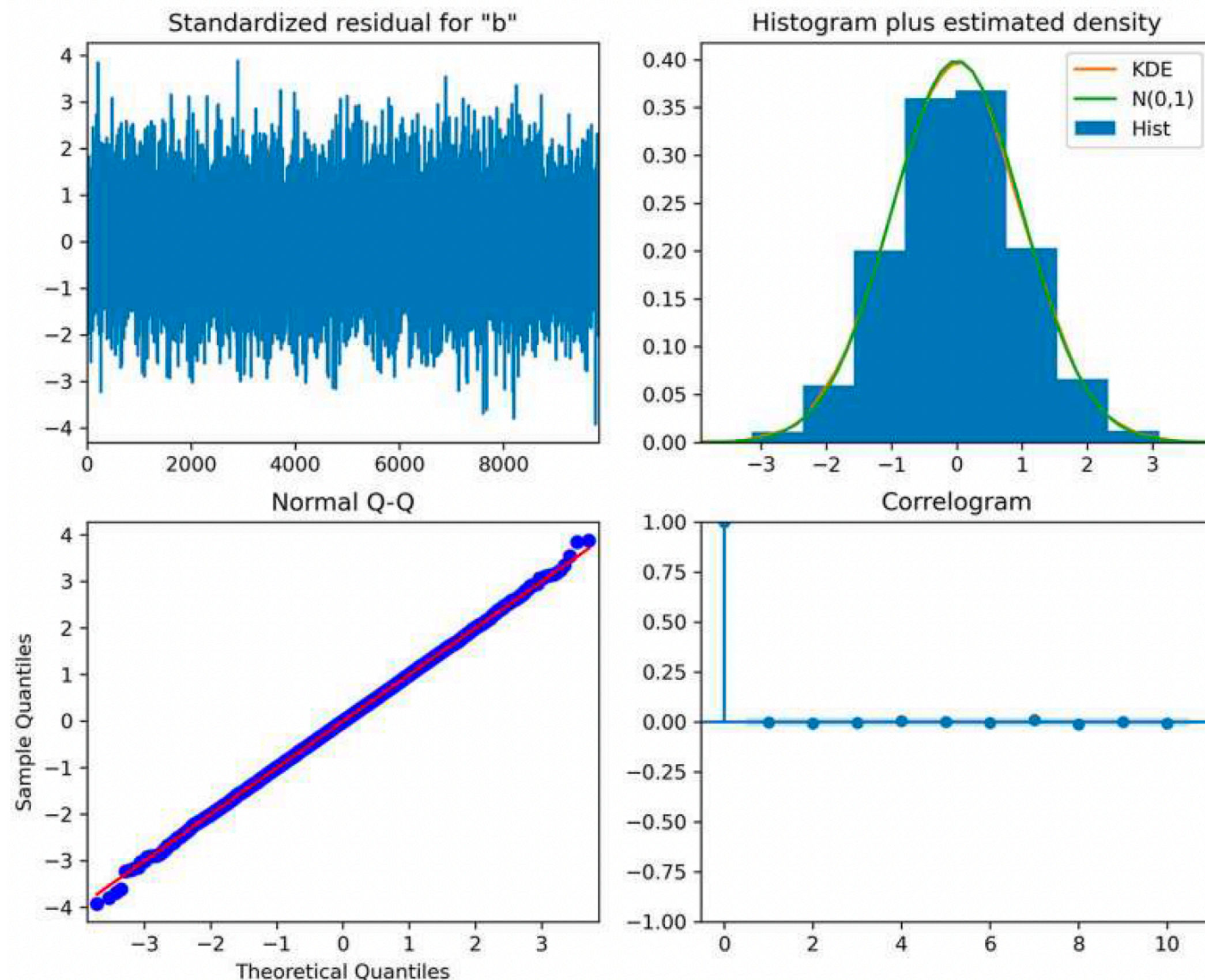
- Our last step is to run the Ljung-Box test on the residuals for the first 10 lags. If the returned p-values exceed 0.05, we cannot reject the null hypothesis, which means that our residuals are uncorrelated and independently distributed, just like white noise.

residuals = model_fit.resid

***lbvalue, pvalue = acorr_ljungbox(residuals,
np.arange(1, 11, 1))***

print(pvalue)

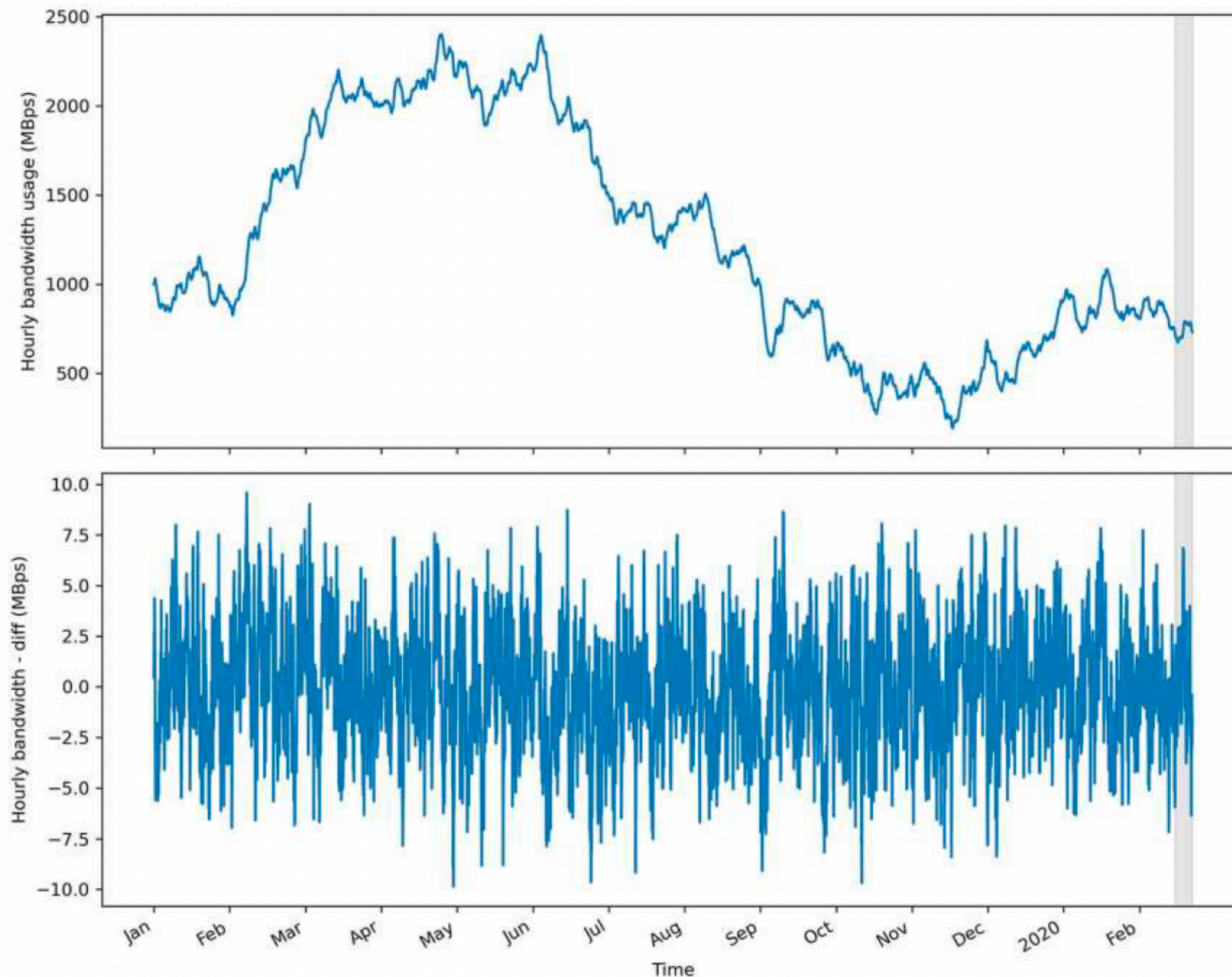
- The returned p-values all exceed 0.05. Therefore, we can conclude that our residuals are indeed uncorrelated. Our ARMA(2,2) model has passed all the checks!



03

Hourly Bandwidth Usage (Forecast)

Forecasting hourly bandwidth usage



- Here we'll keep the last 7 days of data for the test set. Since our forecasts are for the next 2 hours, the test set thus contains 84 periods of 2 hours on which to evaluate our models' performance, since 7 days of hourly data totals 168 hours.

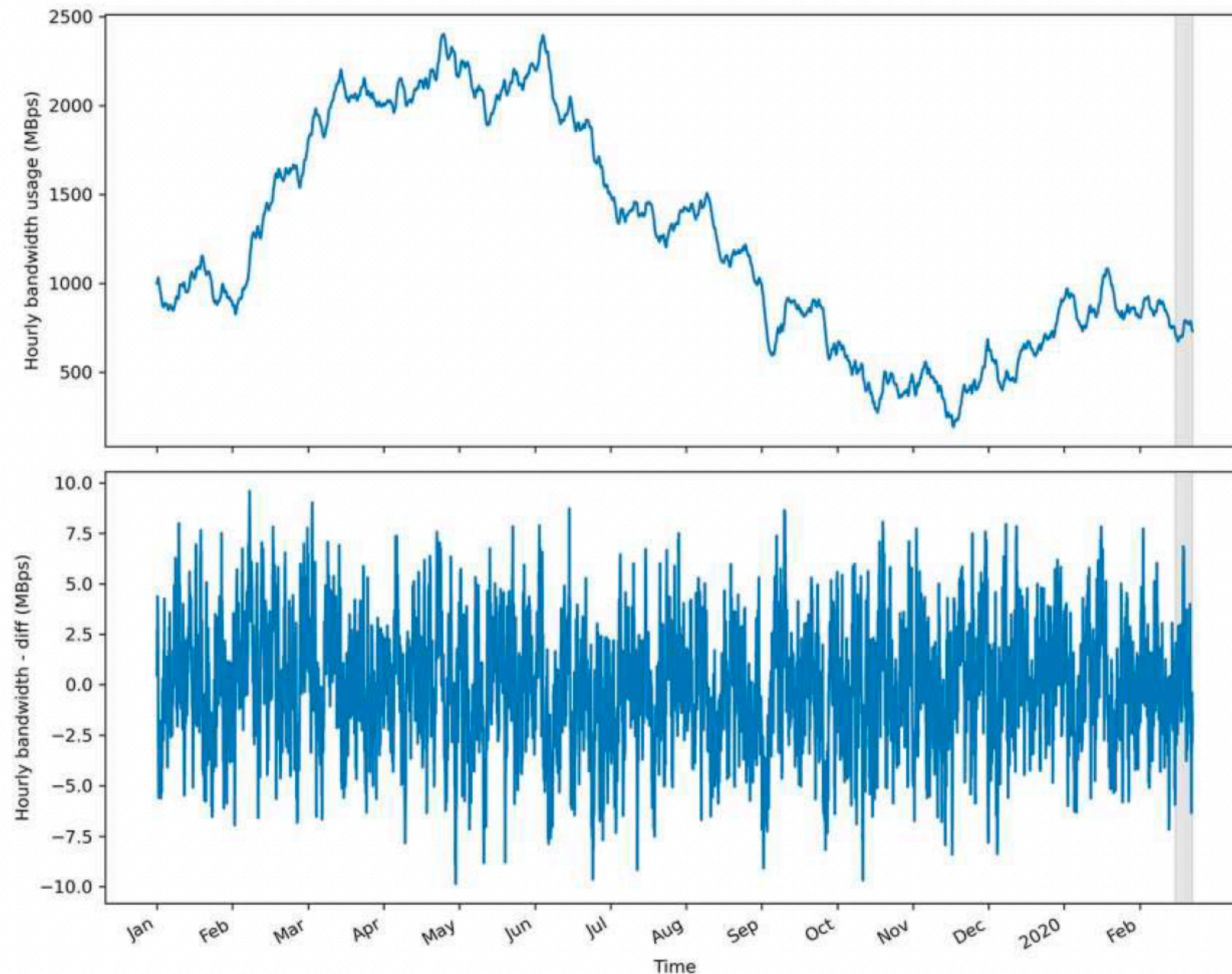
```
df_diff = pd.DataFrame({'bandwidth_diff':  
bandwidth_diff})
```

```
train = df_diff[:-168]
```

```
test = df_diff[-168:]
```

- Note that the sum of both sets gives 9,999, which is one less data point than our original series.

Forecasting hourly bandwidth usage



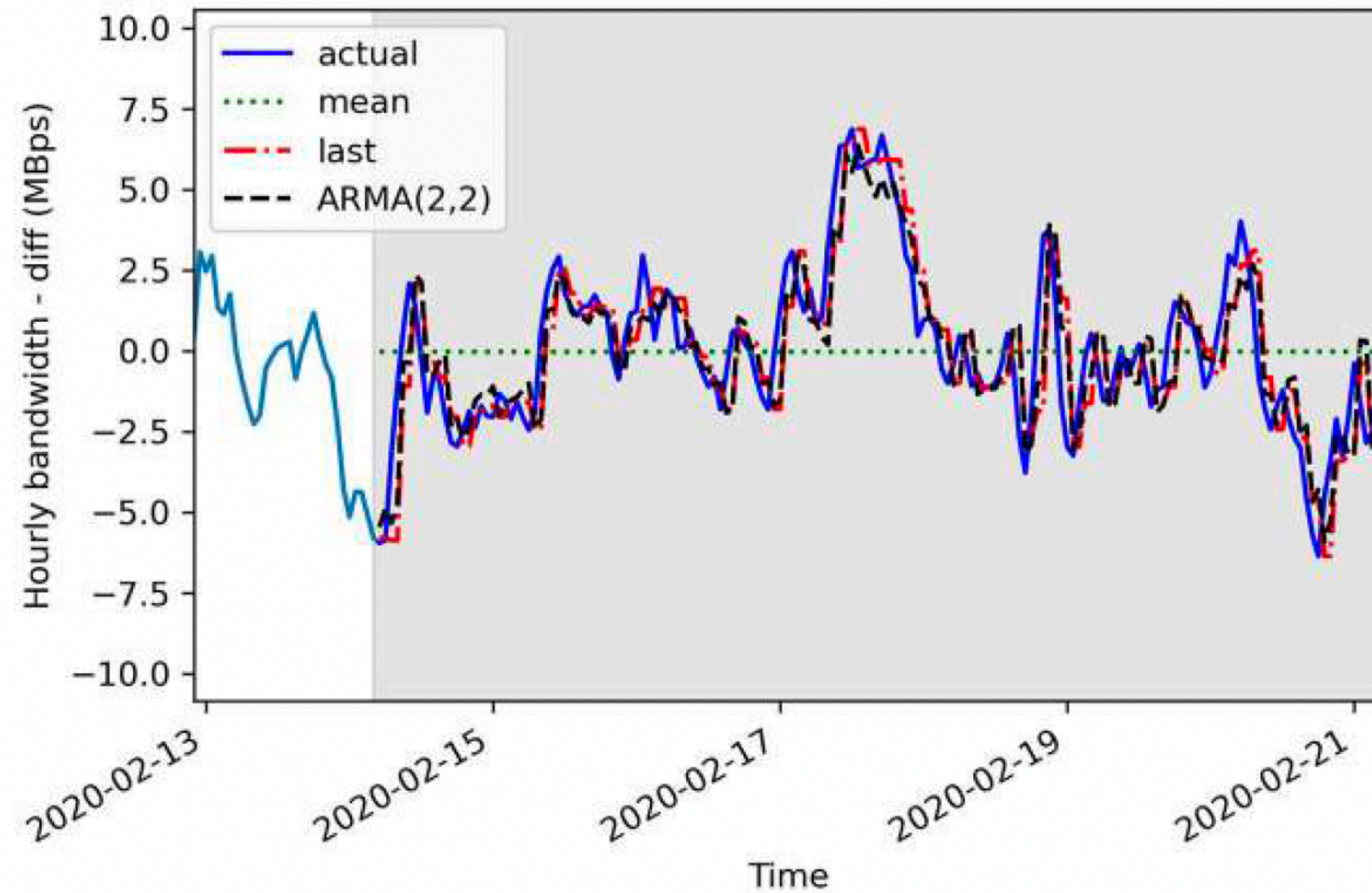
- We can use rolling forecasts:

def rolling_forecast(df: pd.DataFrame, train_len: int, horizon: int, window: int, method: str) -> list:

- Parameters:
 - **train_len** is the number of data points that can be used to fit a model;
 - **horizon** is the length of the test set and represents how many values must be predicted;
 - **window** specifies how many timesteps are predicted at a time;
 - **method** specifies what model to use.

Forecasting hourly bandwidth usage:

Historical Mean



- Each forecasting method is run in a loop. The loop starts at the end of the training set and continues until **total_len**, exclusive, with steps of **window** (**total_len** is the sum of **train_len** and **horizon**):

total_len = train_len + horizon

if method == 'mean':

pred_mean = []

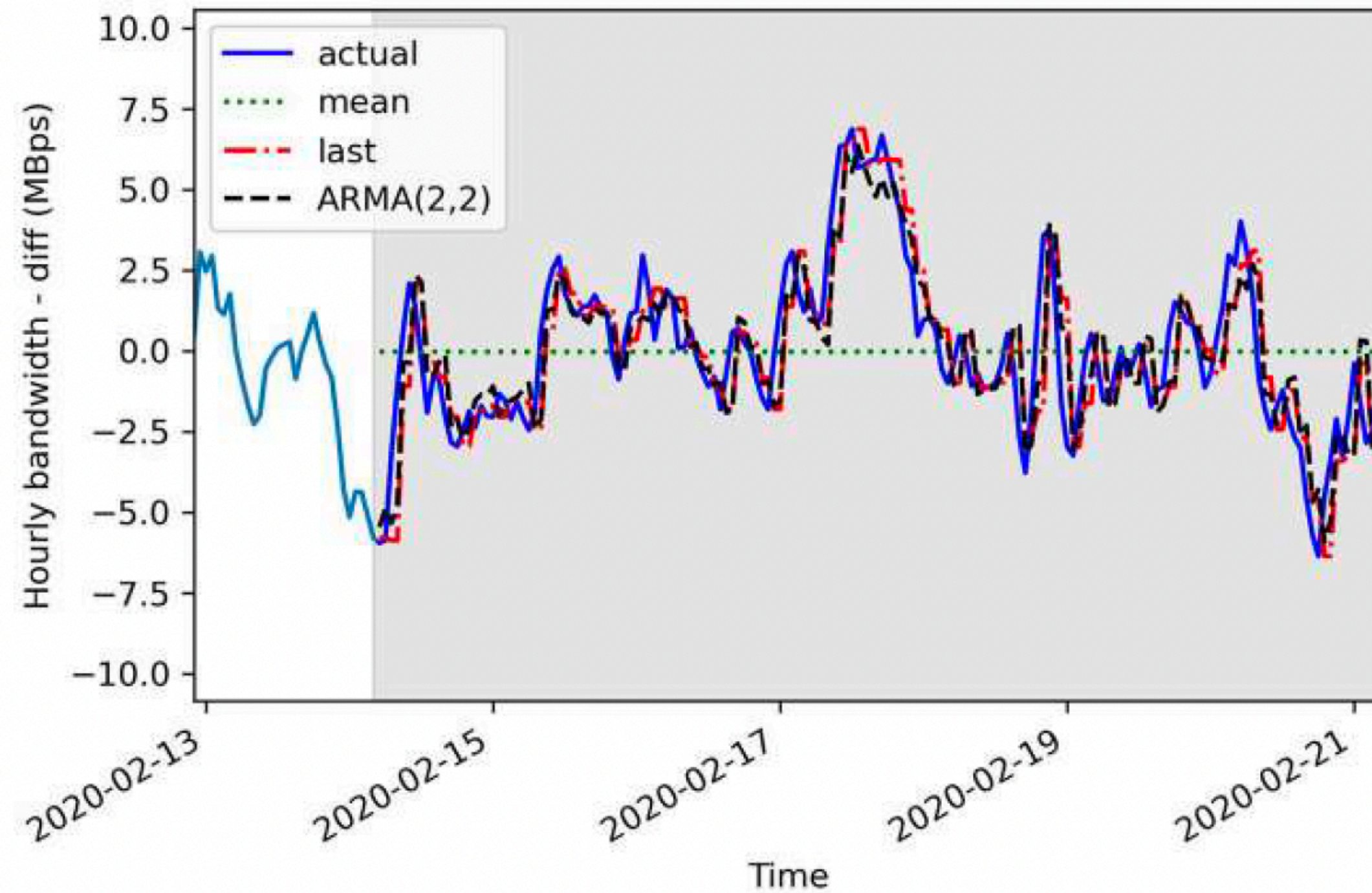
for i in range(train_len, total_len, window):

mean = np.mean(df[:i].values)

pred_mean.extend(mean for _ in range(window))

return pred_mean

Forecasting hourly bandwidth usage: Last Known Value



- Another possible baseline is to predict the last known value of the train set:

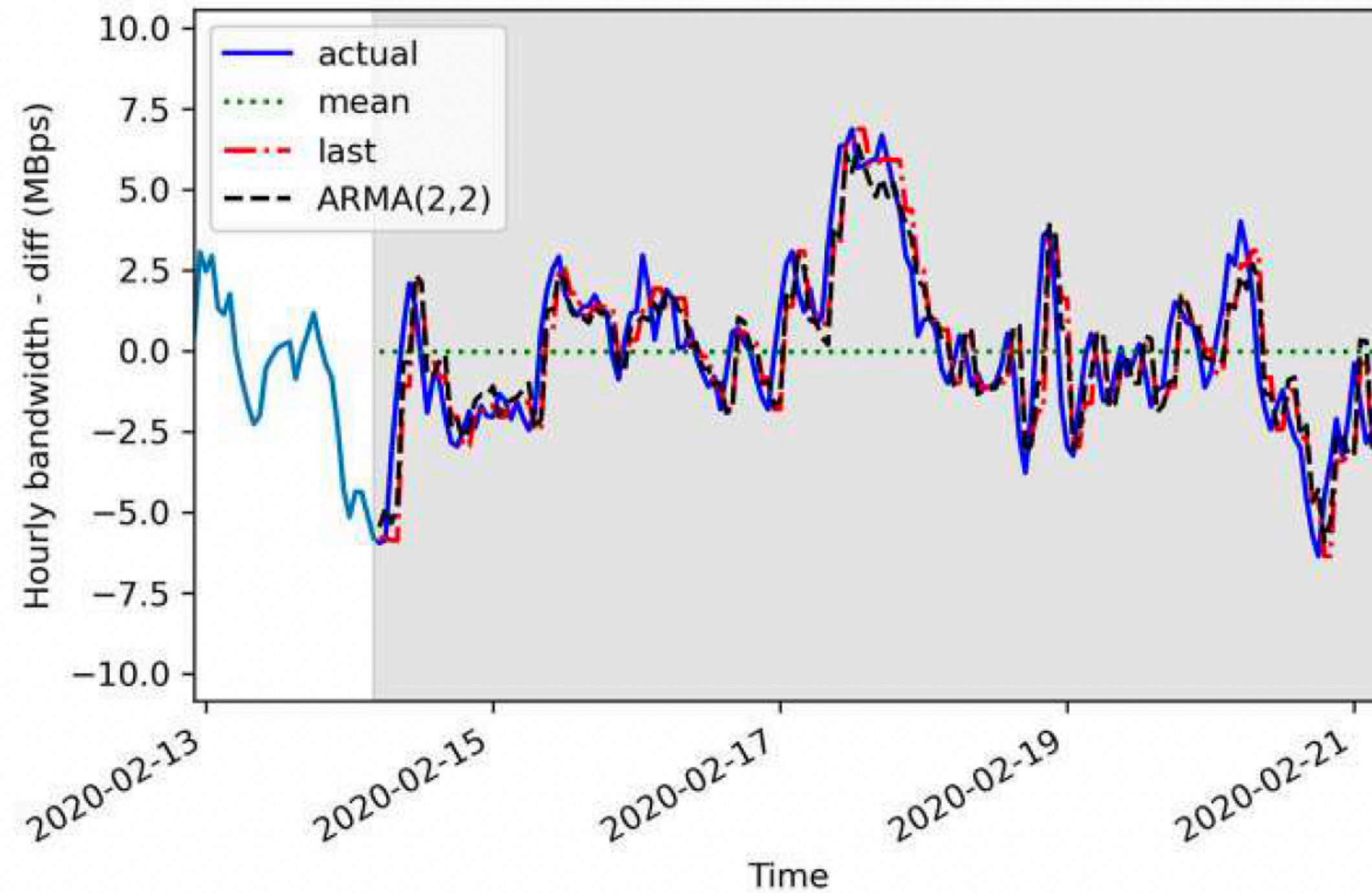
```
elif method == 'last':  
    pred_last_value = []
```

```
for i in range(train_len, total_len, window):  
    last_value = df[:i].iloc[-1].values[0]  
    pred_last_value.extend(last_value for _  
                           in range(window))
```

```
return pred_last_value
```

Forecasting hourly bandwidth usage:

ARMA(2,2)



- Import the **SARIMAX** function from the **statsmodels** library:

```
from statsmodels.tsa.statespace.sarimax import SARIMAX
```

- This function fits an ARMA(2,2) model to our differenced series. Note that SARIMAX is a complex model that allows to consider seasonal effects, autoregressive processes, non-stationary time series, moving average processes, and exogenous variables all in a single model.

```
elif method == 'ARMA':
```

```
    pred_ARMA = []
```

```
    for i in range(train_len, total_len, window):
```

```
        model = SARIMAX(df[:i], order=(2,0,2))
```

```
        res = model.fit(dispatch=False)
```

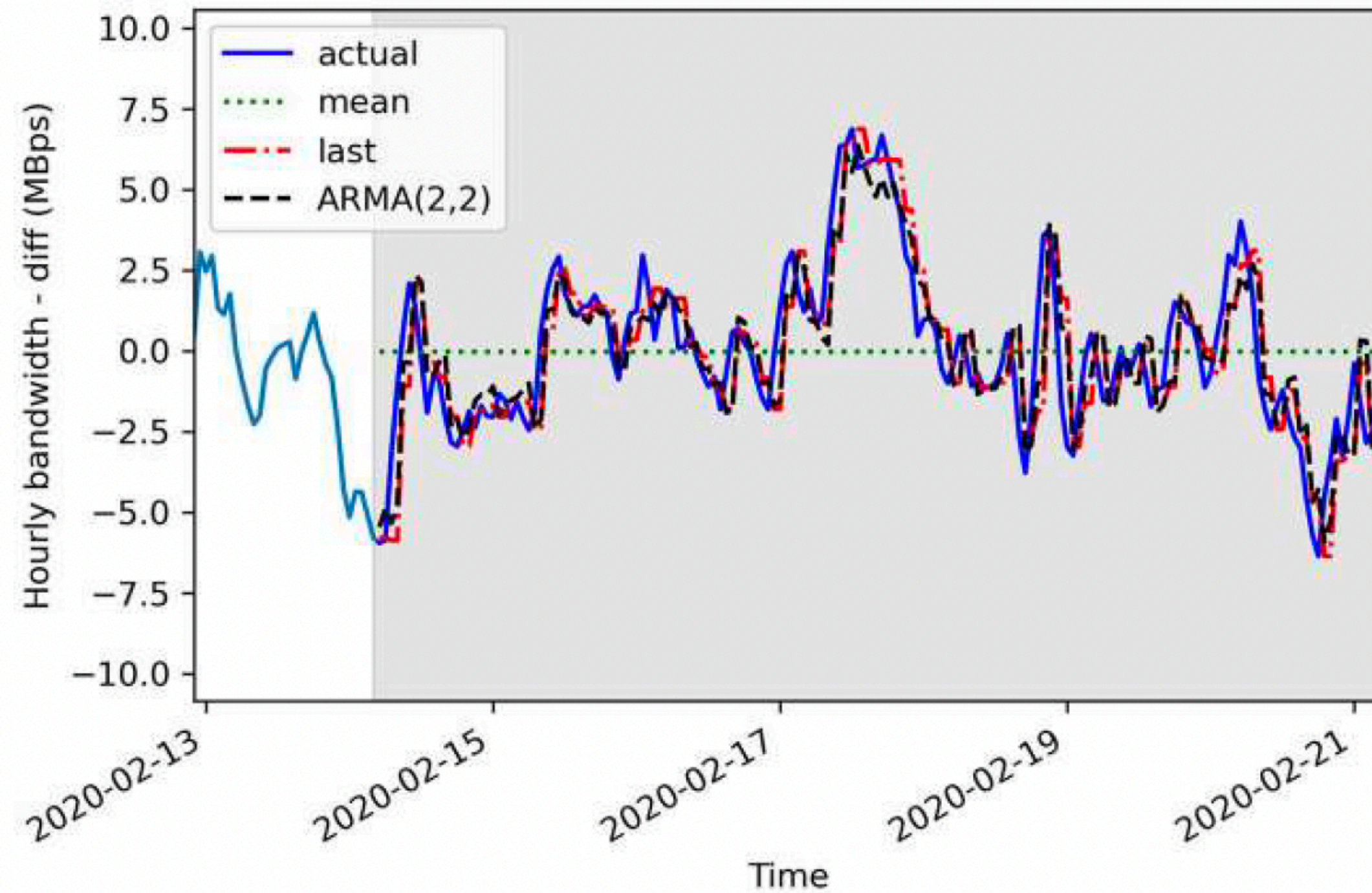
```
        predictions = res.get_prediction(0, i + window - 1)
```

```
        pred = predictions.predicted_mean.iloc[-window:]
```

```
        pred_ARMA.extend(pred)
```

```
    return pred_ARMA
```

Forecasting hourly bandwidth usage



- Use our **rolling_forecast** function to generate a list of predictions for each method. Each list of predictions is then stored in its own column in **pred_df**:

```
pred_df = test.copy()
```

```
TRAIN_LEN = len(train)
```

```
HORIZON = len(test)
```

```
WINDOW = 2
```

```
pred_mean = rolling_forecast(df_diff,  
                             TRAIN_LEN, HORIZON, WINDOW, 'mean')
```

```
pred_last_value = rolling_forecast(df_diff,  
                                   TRAIN_LEN, HORIZON, WINDOW, 'last')
```

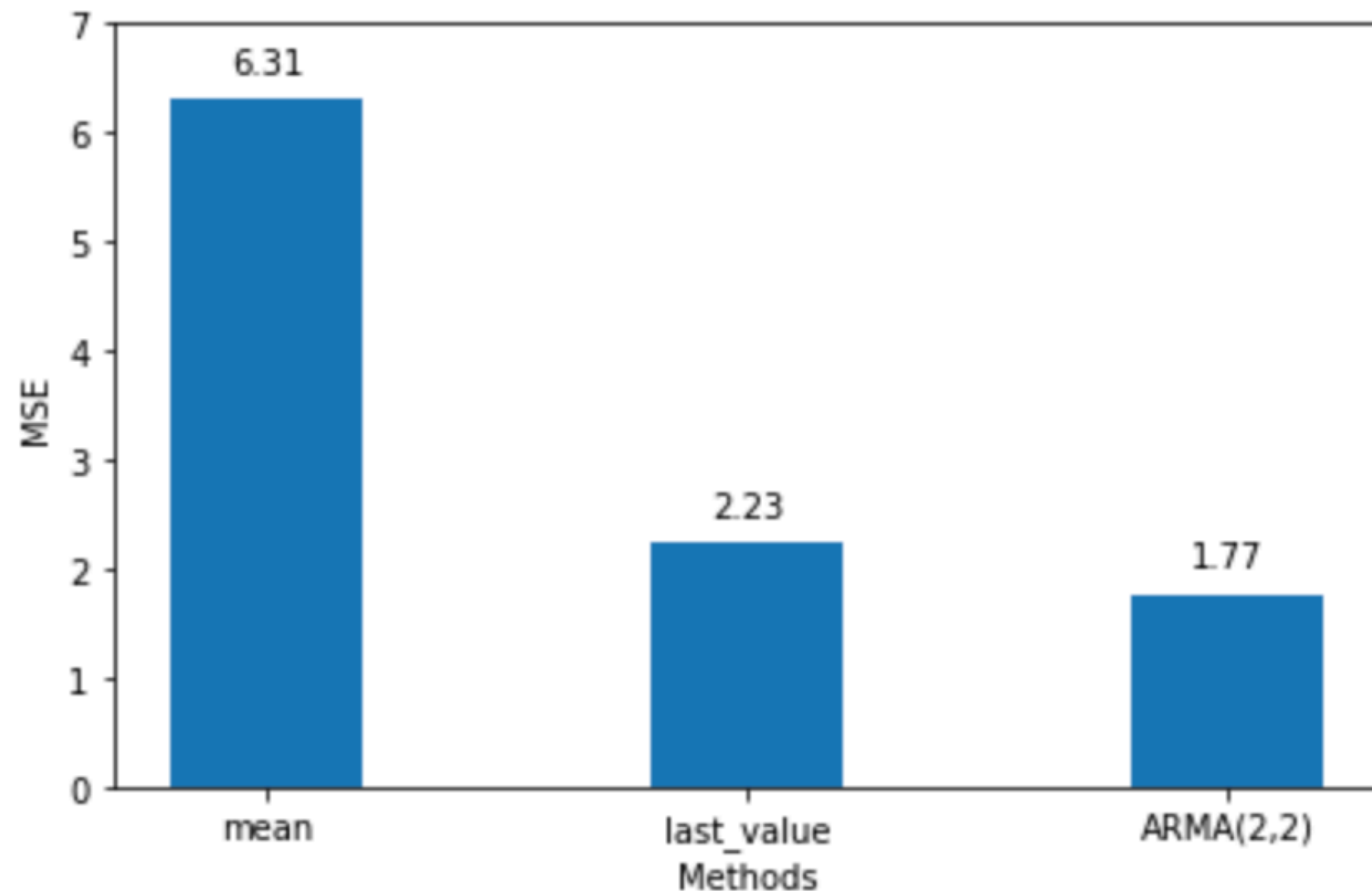
```
pred_ARMA = rolling_forecast(df_diff,  
                             TRAIN_LEN, HORIZON, WINDOW, 'ARMA')
```

```
pred_df['pred_mean'] = pred_mean
```

```
pred_df['pred_last_value'] = pred_last_value
```

```
pred_df['pred_ARMA'] = pred_ARMA
```

Forecasting hourly bandwidth usage (Comparison)



- Measure the MSE:

```
from sklearn.metrics import mean_squared_error
```

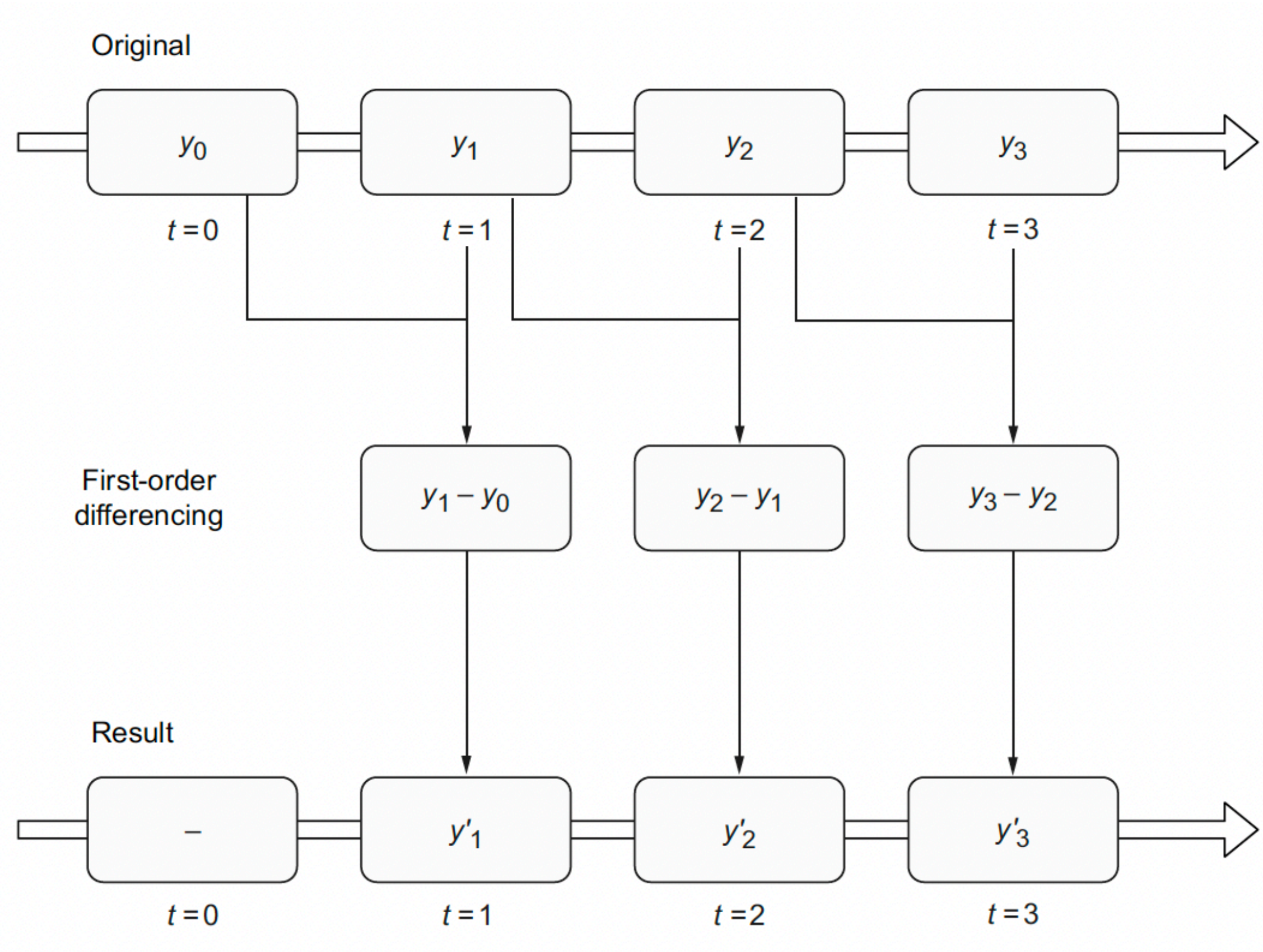
```
mse_mean = mean_squared_error(test['bandwidth_diff'],  
test['pred_mean'])
```

```
mse_last = mean_squared_error(test['bandwidth_diff'],  
test['pred_last_value'])
```

```
mse_MA = mean_squared_error(test['bandwidth_diff'],  
test['pred_ARMA'])
```

- This returns an MSE of 6.3 for the mean method, 2.2 for the last known value method, and 1.8 for the ARMA(2,2) model.
- The ARMA(2,2) model outperforms the benchmarks, meaning that we have a well-performing model.

Reversing the First-Order Difference



- To recover the initial values from the differenced values $y'_t = y_t - y_{t-1}$, we can use the relations:

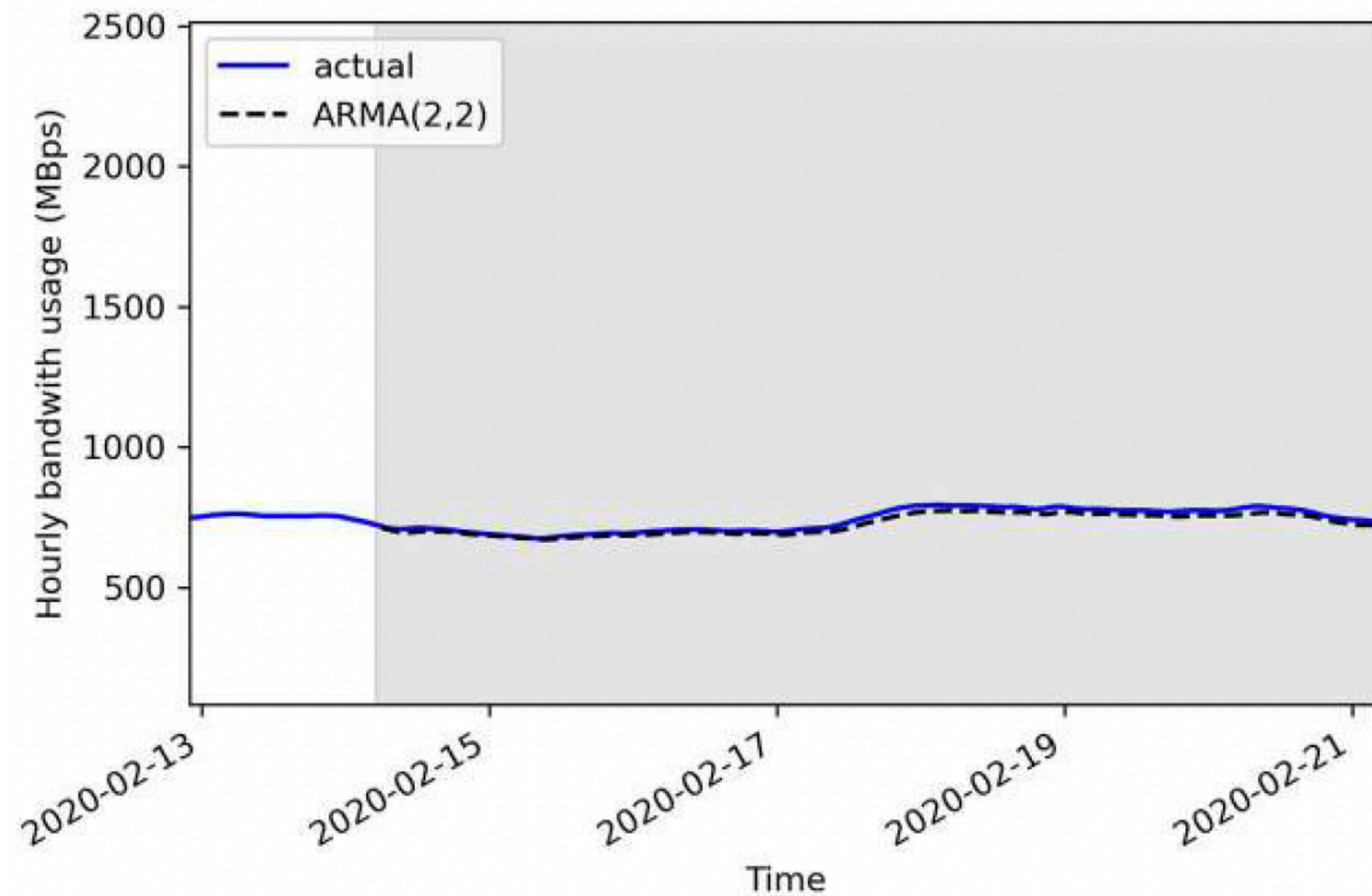
$$y_1 = y_0 + y'_1$$

$$y_2 = y_0 + y'_1 + y'_2$$

...

$$y_t = y_0 + y'_1 + y'_2 + \dots + y'_t$$

Forecasting hourly bandwidth usage: Reversing

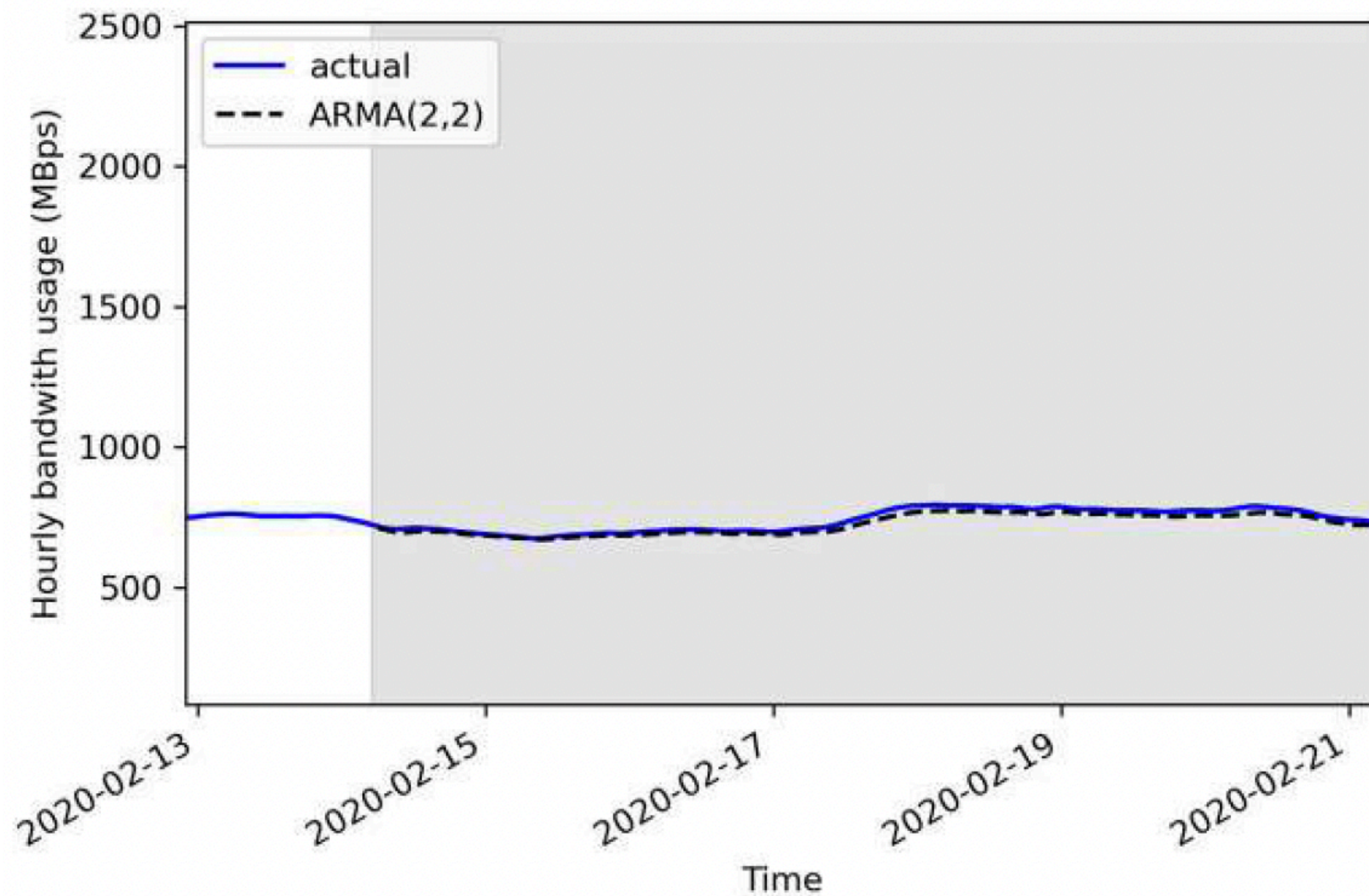


- Since our forecasts are differenced values, we need to reverse the transformation in order to bring our forecasts back to the original scale of the data; otherwise, our predictions will not make sense in a business context.
- To do this, we can take the cumulative sum of our predictions and add it to the last value of our training set in the original series:

```
df['pred_bandwidth'] = pd.Series()
```

```
df['pred_bandwidth'][9832:] =  
df['hourly_bandwidth'].iloc[9832] +  
pred_df['pred_ARMA'].cumsum()
```

Forecasting hourly bandwidth usage: MAE



- We can measure the mean absolute error (MAE) using the **mean_absolute_error** function from **sklearn**. This metric is easy to interpret — it returns the average of the absolute difference between the predicted and actual values, instead of a squared difference like the MSE:

```
mae_ARMA_undiff =  
mean_absolute_error(df['hourly_bandwidth'][9832:],  
df['pred_bandwidth'][9832:])
```

```
print(mae_ARMA_undiff)
```

- This returns an MAE of 14, meaning that, on average, our forecasts are 14 MBps above or below the actual bandwidth usage.

04

Autoregressive Integrated Moving Average Process (ARIMA)

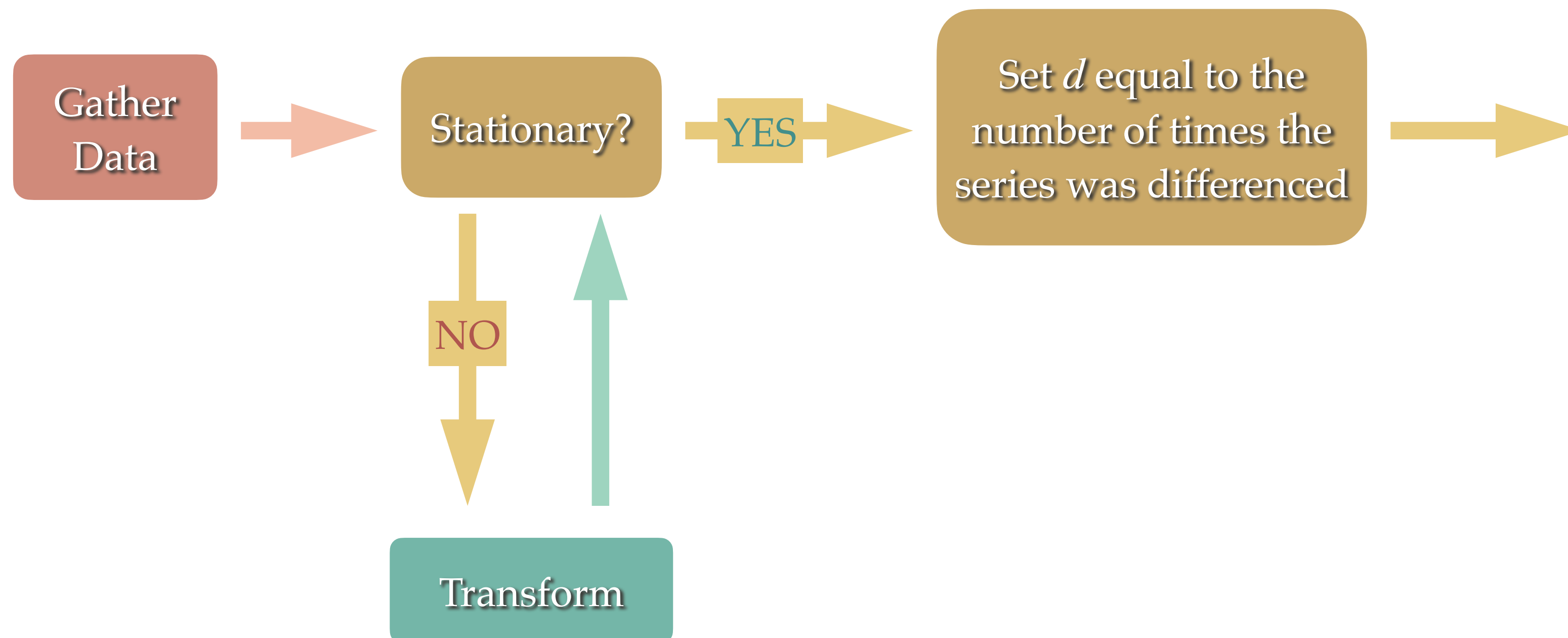
^{def} What is an Autoregressive Moving Average Process (ARMA)?

- ❖ An autoregressive integrated moving average (ARIMA) process is the combination of the $AR(p)$ and $MA(q)$ processes, but in terms of the differenced series.
- ❖ It is denoted by $ARIMA(p, d, q)$, where p is the **order** of the autoregressive part, d is the **order of integration**, and q is the **order** of the moving average part.
- ❖ Integration is the reverse of differencing, and the **order of integration** d is equal to the number of times the series has been differenced to be rendered stationary.
- ❖ The general expression is

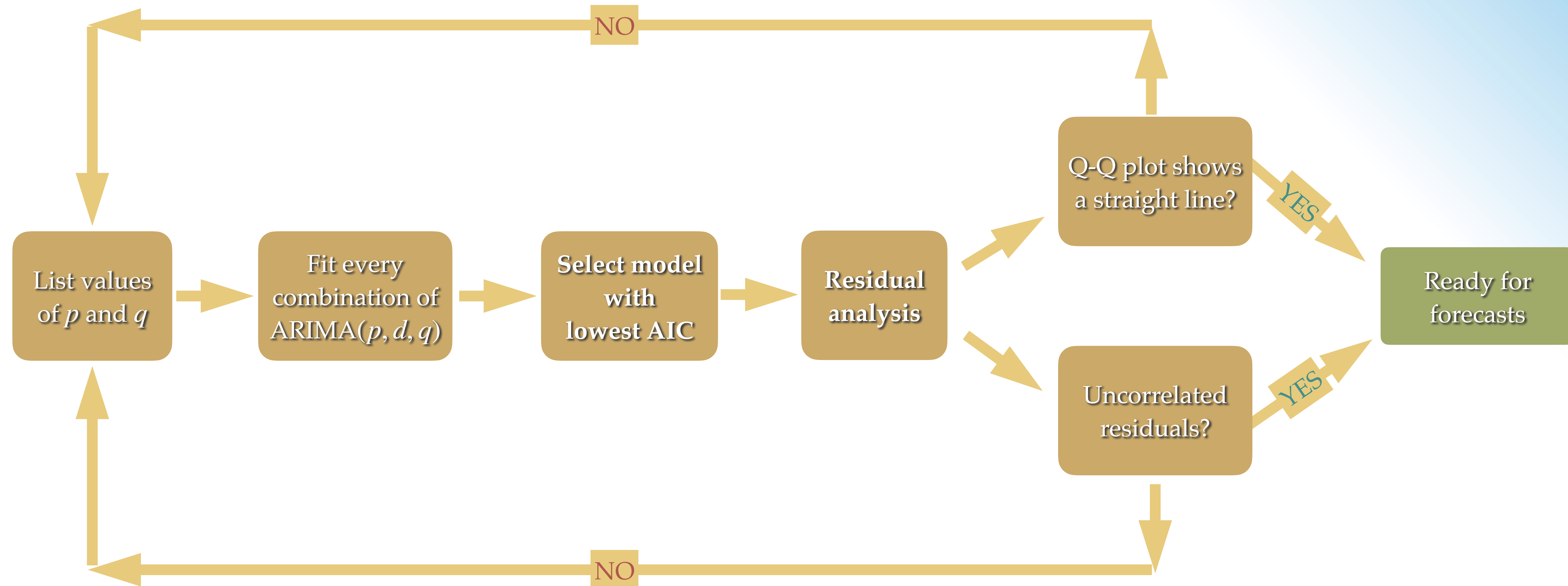
$$y'_t = C + \phi_1 y'_{t-1} + \phi_2 y'_{t-2} + \cdots + \phi_p y'_{t-p} + \epsilon_t + \theta_1 \epsilon_{t-1} + \theta_2 \epsilon_{t-2} + \cdots + \theta_q \epsilon_{t-q}$$

where y'_t represents the differenced series, and it may have been differenced more than once.

Identifying an Autoregressive Integrated Moving Average Process



Identifying an Autoregressive Integrated Moving Average Process



Thank you!